

Doc. no.: P0511R0

Date: 2016-11-09

Audience: Library Evolution Working Group

Reply-to: Zhihao Yuan <zy at miator dot net>

Deduction guide for `std::array`

Abstract

[US 148] suggests to add the following deduction guide to `std::array`

```
template <class T>
array(T&&...) -> array<common_type_t<T...>, sizeof...(T)>;
```

, which is similar to `make_array`’s behavior as presented in [N4600](#) Library Fundamentals V2. This paper suggests that rather than deducing to a “common type”, same-decayed type should be required. Meaning,

```
array{ 1, 2u }
```

will be banned, and

```
array{ "literal", ref_char_ptr }
```

will be preserved.

Discussion

The author of the `make_array` proposal is aware of several drawbacks with the “common type” behavior and put efforts in the adopted facility to address them. For example, an array of common type can be fragile. The deduced “common type” is often unintended, and a later added element can change the type of the entire array, and this change may not be captured at compile-time. When a user runs into these issues, he/she can switch to `make_array<T>`, which simply specifies the element type without deduction, but this option is not available to the deduction guide for `array` since we decided not to deduce a *template-id*.

The trait `common_type_t` can also be fragile. A typical example is that, given the deduction guide specified in [US 148] unmodified,

```
array{ ref(lv_int), ref(lv_int) }
```

gives `array<reference_wrapper<int>>` which is not bad, but

```
array{ ref(lv_int), ref(lv_long) }
```

will give you an `array<long>` that is not even mentioning `reference_wrapper`, and that’s one of the major reasons why `make_array` banned `reference_wrapper`. Given the discussion happened in LEWG, we are not sure whether we want to unwrap references with `tuple` and `pair`’s deduction guides and I’m not sure whether I want to do anything special to `array`’s deduction guide in this area.

The “common type” semantics does not present in other tuple-like types. To rephrase this in an unfortunately academic way, the other two product types, namely `tuple` and `pair`, are not trying to promote type arguments to other types with larger ranges and end up with creating types that are no longer product types. A product type such as `tuple<X, Y>` is called product type because the possible range of values, denoted as $|\text{tuple}<X, Y>|$, is $|X| \times |Y|$, not $|\text{wat}|^2$.

Last but not least, it is quite obvious that

```
vector{ 1, 2.0, 3u }
```

does not give you a `vector<double>`, so why expect `std::array` to do so given exactly the same syntax?

Wording

This wording is relative to N4606.

Add the following signature to 23.3.7.1 [array.overview]/3:

```
constexpr const T * data() const noexcept;
```

```
};
```

```
template<class... T>
```

```
array(T&&...) -> array<see below, sizeof...(T)>;
```

```
}
```

Add the following as paragraph 2 to 23.3.7.2 [array.cons]:

```
template<class... T>
```

```
array(T&&...) -> array< $\mathcal{D}$ , sizeof...(T)>;
```

Remarks: $\mathcal{D}$ is determined as follows: Let U_i be `decay_t< $T_i$ >` for each T_i in τ . Then $\mathcal{D}$ is U_0 and `is_same_v< $\mathcal{D}$ ,  $U_0$ >` is true for all i . If there is no such $\mathcal{D}$, the program is ill-formed.

Implementation

Tested with gcc-7.0.0: [Wandbox](#).